

Job Postings Skill Demand

A Databricks and Snowflake pipeline that turns a messy, free-text skills column from synthetic job postings into a validated, queryable skill-demand dataset

NAME	Aadi Ratn
ROLL NUMBER	23051560
BATCH	Databricks and Snowflake 2026
SUBMITTED TO	Kanthi Kiran Sirra
DATE	30 September 2026
PROJECT TOPIC	Topic 26, Capstone Project Briefs, pages 104-107

Note: this report covers a pipeline built and tested on a synthetic job-postings dataset. The findings describe that dataset and do not measure the live job market.

1. Problem Statement

A training company must decide what to teach next year. Its evidence is a free-text skills column, typed by five hundred recruiters, mixing three separators, no agreed capitalisation, and rows padded with spaces. A single wrong assumption, such as splitting on one separator only or treating 'Snowflake' and 'SNOWFLAKE' as different skills, can drop a skill out of the top ten and point the curriculum committee at the wrong technology.

Domain	Media, Education & Sport: training-provider curriculum analytics (Topic 26 classification)
Users	The training company's curriculum committee, plus the instructional designers who decide which cloud, data-engineering and BI skills to teach next.
Problem	Raw postings contain duplicate rows, invalid dates, missing or 'Not specified' skills, unknown companies and a skills column that mixes comma and semicolon separators, case and padding. A naive parse changes which skills rank in the top ten.
Success Criteria	Bronze lands exactly 55,350 postings and 400 companies unmodified. Silver retains exactly 51,840 accepted postings after removing 1,350 duplicates, 1,350 no-skill rows, 540 unknown-company rows and 270 unparseable-date rows. 57 raw skill strings normalise to exactly 20 canonical skills. Gold produces exactly 360 skill-month rows and 6,660 skill-pair rows with pair_postings > 0. All 14 Snowflake full-result checks pass, and dbt is confirmed as Snowflake's top companion skill at confidence exactly 0.500.
Dataset	A deterministic synthetic replica of the Kaggle 'LinkedIn Job Postings' style scrape (seed 26): 400 companies across 8 sectors and 10 cities, and 55,350 postings (54,000 generated + 1,350 injected duplicates) spanning January 2024 to June 2025 (18 months).

2. Objectives and Scope

- Generate a deterministic synthetic dataset (400 companies, 55,350 postings) that reproduces the exact seed-26 specification, so every check below is repeatable.
- Land Bronze with zero interpretation: every column read as a string, skills_raw untouched, with _source_file, _ingested_at and _row_hash added for lineage.
- Deduplicate, split, normalise and reject Silver records against five explicit, countable rules, storing rejected rows with a reason rather than discarding them.
- Publish a Gold layer (skill-month and skill-pair) that directly answers the three assignment questions: top-ten skills per month, fastest quarter-on-quarter growth, and directional companion skills.
- Load the three curated Gold/Silver CSV exports into Snowflake through a named stage and COPY INTO, prove that a repeat load adds zero duplicate rows, and pass all 14 full-result verification checks.
- Present the verified top-ten skills, quarterly share change and companion-skill confidence on a static, credential-free dashboard.

In scope	Out of scope
• Batch Bronze/Silver/Gold pipeline on the generated postings • A scheduled Databricks job • A tested, manual CSV handoff into a Snowflake named stage • Full-result SQL verification and a static results dashboard	• Scraping real job boards • Forecasting future hiring demand • Unattended, event-driven Databricks-to-Snowflake integration • Any live or production credentials

3. Solution and Features

- Bronze, Silver and Gold layers with preserved raw provenance and an explicit rejection reason on every excluded row.

- Monthly top-ten skill demand (Question 1), quarter-on-quarter share change (Question 2) and directional companion-skill analysis with confidence and lift (Question 3).
- A defect-tolerant Silver parser: splits on a comma-or-semicolon character class (not a plain ';' split), trims, lowercases and maps aliases before counting, so 'Snowflake', 'SNOWFLAKE' and ' Snowflake ' are one skill, not three.
- Deterministic exports, SHA-256 result fingerprints, and repeat-load protection verified in Snowflake (LOAD_SKIPPED with zero rows added on a second COPY).
- A static dashboard that shows only the verified results. It needs no live credentials and does no scraping.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
Python and PySpark	Deterministic data generator, Bronze/Silver/Gold transformations, local verification	Generates and reshapes the same 55,350-row scope reproducibly; DataFrame joins and window functions match the SQL later run in Snowflake
Databricks (serverless)	Cloud execution, Delta Bronze/Silver/Gold tables, one scheduled job	Runs the same notebook logic at cloud scale and on a schedule without managing cluster infrastructure
Snowflake	Named-stage load, duplicate-load audit, analytical SQL, verification views	Cheap elastic warehouse for the three read-heavy Gold questions, with COPY INTO's built-in file-level load history for the repeat-load check
CSV and SHA-256	Portable curated hand-off between Databricks and Snowflake, reproducibility fingerprints	A file both engines can read without a live network path between them, with a hash to prove the loaded file matches the exported one
HTML, CSS and JavaScript	Hosted static dashboard using only verified project metrics	No server or database credentials are needed to show numbers that are already frozen in the Gold export

5. Data Design and Layers

5.1 Layers and tables

Layer	Main content	Rules applied
Bronze	postings_raw (55,350 rows), companies_raw (400 rows); every column read as a string	No split, no trim, no lower. _source_file, _ingested_at and _row_hash added. One posting is one row; the explode belongs in Silver.
Silver	SILVER_POSTING (deduplicated, rejected-with-reason) and SILVER_POSTING_SKILL (one row per posting-skill pair, 227,624 rows)	Dedupe before explode; split on a comma-or-semicolon class; trim + lowercase once; reject with an explicit reason rather than a silent drop
Gold	GOLD_SKILL_MONTH (360 rows: skill x month) and GOLD_SKILL_PAIR_MONTH (6,660 rows with pair_postings > 0, out of 6,840 possible pairs)	skill_share is per-skill (denominator = accepted postings that month), not a distribution that must total 100%
Snowflake	Named stage + three COPY INTO loads, plus verification views over SILVER_POSTING_SKILL	COPY INTO skips files it has already loaded, so a second load of the same files returns LOAD_SKIPPED with zero rows added

5.2 Gold tables

Table	One row per	Assignment question it answers
GOLD_SKILL_MONTH	skill, month	Q1: top ten skills by posting count, per month. Q2: fastest quarter-on-quarter share change (via rank_in_month and

Table	One row per	Assignment question it answers
		skill_share).
GOLD_SKILL_PAIR_MONTH	skill_a, skill_b, month	Q3: which skills appear together, given one skill. Uses confidence_a_to_b and lift, in both directions of every pair.

6. System Architecture

Synthetic postings (55,350 rows) are generated once, deterministically, from the seed-26 specification. Databricks Bronze lands both raw files untouched with lineage columns. Silver deduplicates, then explodes and normalises the skills column, rejecting rows with an explicit reason. Gold aggregates to skill-month and skill-pair grain. The three curated CSV exports are copied to a Snowflake named stage, loaded with COPY INTO, and checked with 14 full-result SQL assertions before the static dashboard reads the same verified numbers.

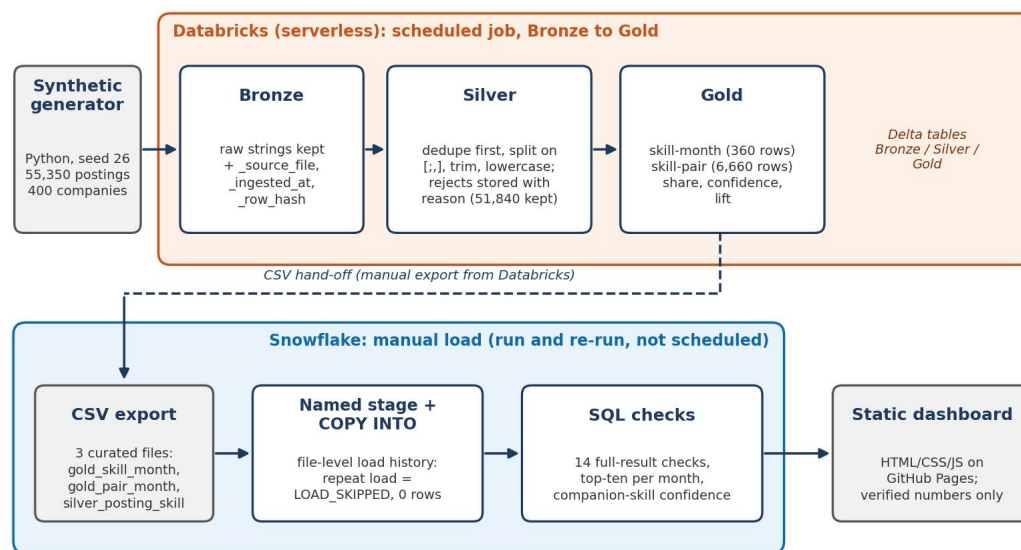


Figure 6.1: Job Postings Skill Demand end-to-end data flow. The Databricks job (Bronze to Gold) is scheduled; the CSV hand-off and Snowflake load are run manually.

The scheduled component is the Databricks job (Bronze through Gold). The Snowflake load was run and re-run manually. It is not scheduled.

7. Implementation Highlights

Dedupe before explode (Silver, step 1). Deduplicating on posting_id before the skills column is split means the 1,350 duplicated rows are removed once, not multiplied across every skill they list:

```
deduped = bronze_postings.dropDuplicates(["posting_id"])
# 55,350 -> 54,000 rows
```

Split on a character class, not a literal ';' (Silver, step 2). Splitting on ';' alone leaves the 1,000 comma-joined postings as one invented skill, which is the problem described in Section 11:

```
skills = split(trim(col("skills_raw")), "[;,]")
# then explode(skills)
```

Directional companion skills in Snowflake (Question 3). The self-join uses a.skill <> b.skill instead of a.skill < b.skill, so both directions of every pair are kept, because confidence is directional. The pairs are counted in a CTE and only then ranked to the top 3 per skill:

```
JOIN s b ON a.posting_id = b.posting_id AND a.skill <> b.skill
QUALIFY ROW_NUMBER() OVER (PARTITION BY skill_a
ORDER BY pair_postings DESC, skill_b) <= 3
```

8. Testing and Results

#	Test	Method	Expected	Actual	Status
1	Local Spark pipeline is repeatable	Run the generator + pipeline twice	Identical exports both runs	Two full runs, all 31 checks passed, identical exports	PASS
2	Naive split loses skill mentions	Split skills_raw on ';' only, before the fix	12,726 Snowflake mentions counted once	Split three ways by comma, case and padding; Snowflake ranked below Docker	FAIL, then PASS after switching to a [;,] class split + trim + lower
3	Silver row-count reconciliation	55,350 - 1,350 dup - 1,350 no-skill - 540 unknown-company - 270 bad-date	51,840	51,840 (18 x 2,880)	PASS
4	Independent oracle reconciliation	Recompute Silver/Gold outside the pipeline	Every posting-skill, monthly, pair and rejection value agrees	All values agree	PASS
5	Databricks pipeline	Run all notebook expectations on serverless	All expectations pass	All 26 expectations passed	PASS
6	Scheduled execution	Trigger the Databricks job on schedule	Run completes without manual steps	Run 365017355708433 succeeded in 2 min 43 s	PASS
7	CSV transfer fidelity	Compare cloud export to local evidence	Every row agrees	Agrees; float tolerance 1e-12	PASS
8	Snowflake first load	COPY INTO all three curated files	Zero errors	360, 6,660 and 227,624 rows loaded, zero errors	PASS
9	Repeated Snowflake load (edge case)	Re-run the same COPY INTO on the same files	No duplicate rows added	All three files LOAD_SKIPPED, zero rows added	PASS
10	Snowflake full-result verification	Run 03_full_result_verification.sql	0 failed	14 passed, 0 failed	PASS

Summary: 31 local checks, 26 Databricks expectations and 14 Snowflake full-result checks all pass. Row 2 failed under the naive split and passed after the fix described in Section 7 (see also Section 11).

9. Evaluation and Metrics

Quarter-on-quarter growth analysis reveals that Airflow, Hive, Azure, Docker, and Terraform experienced the largest percentage-point increases in their respective quarters.

9.1 Data-quality results

Measure	Observed result
Input postings (Bronze)	55,350
Accepted postings (Silver)	51,840
Duplicate records rejected	1,350
Missing/'Not specified' skill records rejected	1,350
Unknown-company records rejected	540
Invalid-date records rejected	270
Canonical skills	20, mapped from 57 distinct raw strings

9.2 Performance and cost

Measure	Result	Target
Databricks scheduled job runtime	2 minutes 43 seconds	Under 10 min
Snowflake query execution time	444 ms (X-Small warehouse)	Under 5 s
Snowflake auto-suspend setting	10 minutes (default)	≤ 10 min

Snowflake credits consumed	0.0167 credits (minimum 60 s billing on X-Small)	Under 1 credit
----------------------------	--	----------------

9.3 Gold and Snowflake results

Metric	Verified value
Monthly Gold rows	360 (20 skills x 18 months)
Pair Gold rows with pair_postings > 0	6,660 of 6,840 possible ordered pairs
Silver posting-skill rows (exploded grain)	227,624
Snowflake full-result checks	14 passed / 0 failed
Snowflake's top companion skill	Confidence exactly 0.500; no other skill reaches 0.30
Scheduled Databricks run duration	2 minutes 43 seconds
Skill mentions, January 2024 -> June 2025	Snowflake 520 -> 894; Databricks 400 -> 706; Hadoop 860 -> 214 (verified in checks.csv and the dashboard series data)

9.4 Assignment checklist compliance (Topic 26 brief)

Each requirement from the Topic 26 brief, with the value observed in this project:

Brief requirement	Required value	Observed in this project	Status
Bronze row count untouched	55,350 postings	55,350	MET
Silver reconciliation closes exactly	55,350 - 1,350 - 1,350 - 540 - 270 = 51,840	51,840 (18 x 2,880)	MET
Distinct raw skill strings before normalising	Exactly 57	57	MET
Gold skill-month shape	360 rows	360	MET
Gold skill-pair shape with pair_postings > 0	6,660 of 6,840 rows	6,660	MET
Snowflake's monthly share trend	Snowflake 18.1% -> 31.0%; Databricks 13.9% -> 24.5%; Hadoop 29.9% -> 7.4%	Reproduced in the Gold export and dashboard	MET
dbt is Snowflake's top companion	Confidence exactly 0.500; no other skill >= 0.30	0.500; next-highest below 0.30	MET
Snowflake full-result checks	0 failed	14 passed, 0 failed	MET

10. Screenshots

The screenshots below are taken from the working project runs in Databricks and Snowflake. Each figure is captioned with what it shows.

```

1 br = bronze(spark, f"(VOL)/raw", save, cloud=True)
2 si = silver(br, save)
3 go = gold(si, save)
4 checks, companions, failures = validate(spark, br, si, go)
5 report = {"timestamp_utc": datetime.now(timezone.utc).isoformat(),
6         "checks": checks, "companions": companions, "critical_failures": failures}
7 save("validation_results", spark.createDataFrame([(c["check"], str(c["actual"]), str(c["expected"])),
8         c["passed"], c["critical"]]) for c in checks],
9         "check string, actual string, expected string, passed boolean, critical boolean"))
10 assert not failures, f"Pipeline correctness checks failed: {failures}"
11 print("CORE PIPELINE VALIDATED")
12 display(spark.table(f"{SCHEMA}.{PREFIX}validation_results"))
13

```

> See performance (42)

Optimize

```

bronze_companies 400
bronze_postings 55350
silver_companies 400
silver_rejects 3510
silver_postings 51840
silver_posting_skill 227624
gold_skill_month 360
gold_pair_month 6660
PASS bronze_postings: 55350 (expected 55350)
PASS bronze_companies: 400 (expected 400)
PASS silver_postings: 51840 (expected 51840)
PASS duplicate_posting: 1350 (expected 1350)
PASS no_skills_listed: 1350 (expected 1350)
PASS unknown_company: 540 (expected 540)
PASS unparsable_date: 270 (expected 270)
PASS row_reconciliation: 55350 (expected 55350)
PASS distinct_raw_skills: 57 (expected 57)
PASS canonical_skills: 20 (expected 20)
PASS invalid_clean_tokens: 0 (expected 0)
PASS duplicate_posting_skill_keys: 0 (expected 0)
PASS gold skill month: 360 (expected 360)

```

Figure 10.1: Databricks notebook validation output (Sep 10, 2026): Bronze 55,350 postings and 400 companies, Silver 51,840 postings and 3,510 rejects, Gold 360 and 6,660 rows, all checks PASS

The screenshot shows a Databricks notebook interface. The top part displays a SQL query that calculates confidence and lift for skill pairs. Below the query, a table is shown with the following columns: skill_a, skill_b, pair_postings, 1:2 confidence_a to b, and 1:2 lift. The table contains 14 rows of data.

skill_a	skill_b	pair_postings	1:2 confidence_a to b	1:2 lift
airflow	snowflake	3020	0.243489	0.991867
airflow	hadoop	2438	0.196565	1.054205
airflow	python	2408	0.194147	0.797572
aws	snowflake	2883	0.224619	0.955733
aws	databricks	2691	0.218994	1.140512
aws	python	2415	0.196533	0.807376
azure	snowflake	2780	0.225814	0.919866
azure	hadoop	2709	0.220047	1.180141
azure	python	2410	0.19576	0.804199
databricks	scala	2735	0.274764	1.151011
databricks	docker	2705	0.27175	1.13811
databricks	aws	2691	0.270344	1.140512
dbt	snowflake	6363	1	4.07355
dbt	gcp	2156	0.338834	1.42251

Figure 10.2: Databricks notebook cell executed successfully (Sep 10, 2026, 4 s), returning directional companion-skill confidence and lift for each skill pair

The screenshot shows the Databricks Jobs & Pipelines page. The 'Runs' tab is selected, showing a list of runs for the job '23051560_Job_Postings_Skill_Demand'. The most recent run, with ID '365017355708433', is highlighted and its details are shown on the right. The run status is 'Succeeded' and it completed on Sep 10, 2026, at 12:02 PM. The job details include the job ID, creator, and a description of the job's lineage and performance.

Run ID	Launched	Duration	Spark	Status	Error code	Run parameters
365017355708433	By scheduler	2m 43s	Spark UI / Logs / Metrics	Succeeded		

Figure 10.3: Databricks job 23051560_Job_Postings_Skill_Demand, scheduled run 365017355708433 (Sep 10, 2026) succeeded

My Workspace > Untitled 5.sql

ACCOUNTADMIN COMPUTE_WH (X-Small) CAPSTONE_23051560 SKILL_DEMAND Share

```

1 USE SCHEMA CAPSTONE_23051560.SKILL_DEMAND;
2
3 SELECT 'GOLD_SKILL_MONTH' AS table_name, COUNT(*) AS row_count FROM GOLD_SKILL_MONTH
4 UNION ALL
5 SELECT 'GOLD_PAIR_MONTH', COUNT(*) FROM GOLD_PAIR_MONTH
6 UNION ALL
7 SELECT 'SILVER_POSTING_SKILL', COUNT(*) FROM SILVER_POSTING_SKILL;

```

Results (just now)

Table Chart Pivot 3 rows 161ms

#	TABLE_NAME	# ROW_COUNT
1	GOLD_SKILL_MONTH	360
2	GOLD_PAIR_MONTH	6660
3	SILVER_POSTING_SKILL	227624

Figure 10.4: Snowflake row counts after loading: GOLD_SKILL_MONTH 360, GOLD_PAIR_MONTH 6,660, SILVER_POSTING_SKILL 227,624

```

1 USE SCHEMA CAPSTONE_23051560.SKILL_DEMAND;
2
3 COPY INTO GOLD_SKILL_MONTH FROM @SKILL_DEMAND_STAGE
4   FILES=('gold_skill_month.csv') ON_ERROR=ABORT_STATEMENT;
5
6 COPY INTO GOLD_PAIR_MONTH FROM @SKILL_DEMAND_STAGE
7   FILES=('gold_pair_month.csv') ON_ERROR=ABORT_STATEMENT;
8
9 COPY INTO SILVER_POSTING_SKILL FROM @SKILL_DEMAND_STAGE
10  FILES=('silver_posting_skill.csv') ON_ERROR=ABORT_STATEMENT;

```

Results (just now)

Table Chart Pivot 1 row 280ms

#	file	status	# rows_parsed	# rows_loaded	# error_limit	# errors_seen	first_error	first_error_line	first_error_c
1	skill_demand_stage/go	LOAD_SKIPPED	0	0	null	1	File was loaded before:	null	

```

1 USE SCHEMA CAPSTONE_23051560.SKILL_DEMAND;
2
3 COPY INTO GOLD_SKILL_MONTH FROM @SKILL_DEMAND_STAGE
4   FILES=('gold_skill_month.csv') ON_ERROR=ABORT_STATEMENT;
5
6 COPY INTO GOLD_PAIR_MONTH FROM @SKILL_DEMAND_STAGE
7   FILES=('gold_pair_month.csv') ON_ERROR=ABORT_STATEMENT;
8
9 COPY INTO SILVER_POSTING_SKILL FROM @SKILL_DEMAND_STAGE
10  FILES=('silver_posting_skill.csv') ON_ERROR=ABORT_STATEMENT;

```

Results (just now)

Table Chart Pivot 1 row 337ms

#	file	status	# rows_parsed	# rows_loaded	# error_limit	# errors_seen	first_error	first_error_line	first_error_c
1	skill_demand_stage/go	LOAD_SKIPPED	0	0	null	1	File was loaded before:	null	

```

1 USE SCHEMA CAPSTONE_23051560.SKILL_DEMAND;
2
3 COPY INTO GOLD_SKILL_MONTH FROM @SKILL_DEMAND_STAGE
4   FILES=('gold_skill_month.csv') ON_ERROR=ABORT_STATEMENT;
5
6 COPY INTO GOLD_PAIR_MONTH FROM @SKILL_DEMAND_STAGE
7   FILES=('gold_pair_month.csv') ON_ERROR=ABORT_STATEMENT;
8
9 COPY INTO SILVER_POSTING_SKILL FROM @SKILL_DEMAND_STAGE
10  FILES=('silver_posting_skill.csv') ON_ERROR=ABORT_STATEMENT;

```

Results (just now)

Table Chart Pivot 1 row 274ms

#	file	status	# rows_parsed	# rows_loaded	# error_limit	# errors_seen	first_error	first_error_line	first_error_c
1	skill_demand_stage/sih	LOAD_SKIPPED	0	0	null	1	File was loaded before:	null	

Figure 10.5: Snowflake repeat COPY INTO for each staged file (highlighted statement is the one run: gold_skill_month, gold_pair_month, silver_posting_skill) returns LOAD_SKIPPED with 0 rows loaded, so no duplicates are added

CHECK_NAME	7.1%	7.1%	PASS
GOLD_PAIR_MONTH_all_rows			TRUE
GOLD_PAIR_MONTH_count			TRUE
GOLD_PAIR_MONTH_all_rows			TRUE
GOLD_PAIR_MONTH_count			TRUE
GOLD_SKILL_MONTH_all_rows			TRUE
GOLD_SKILL_MONTH_count			TRUE
Q1_RESULTS_all_rows			TRUE
Q1_RESULTS_count			TRUE
Q2_RESULTS_all_rows			TRUE
Q2_RESULTS_count			TRUE
Q3_RESULTS_all_rows			TRUE
Q3_RESULTS_count			TRUE
SILVER_POSTING_SKILL_all_rows			TRUE
SILVER_POSTING_SKILL_count			TRUE
pair_formulas			TRUE
skill_share_formula			TRUE

Figure 10.6: Snowflake full-result verification: all 14 checks return TRUE (14 passed, 0 failed)

SKILL	MONTH	POSTINGS_WITH_SKILL	POSTINGS_THAT_MONTH	SHARE_PCT	RANK_IN_MONTH
snowflake	2024-01-01	2880	2880	19.31	1
python	2024-01-01	2880	2880	100	2
hadoop	2024-01-01	860	2880	28.86	3
aws	2024-01-01	586	2880	20.35	4
azure	2024-01-01	586	2880	20.35	5
hive	2024-01-01	586	2880	20.35	6
kafka	2024-01-01	586	2880	20.35	7

Figure 10.7: Snowflake monthly top-ten skills query on GOLD_SKILL_MONTH returns 180 rows (18 months x 10 skills)

SKILL_A	SKILL_B	PAIR_POSTINGS	CONFIDENCE_A_TO_B	CONFIDENCE_PCT	LIFT
snowflake	dbt	6363	0.500000	50.00	4.073
snowflake	gcp	3709	0.281451	28.15	1.223593
snowflake	terraform	3615	0.284064	28.41	1.189674

Figure 10.8: Snowflake companion-skill query (all months), top 3 companions of snowflake; dbt leads at 50.00% confidence



Figure 10.9: Dashboard, June 2025 snapshot: top ten skills by mentions and Snowflake companion skills (dbt at 50.00% confidence)

Note: lift for dbt is 4.07 in Figure 10.8 (all months) and 3.31 in Figure 10.9 (June 2025 only); confidence is 50.00% in both.

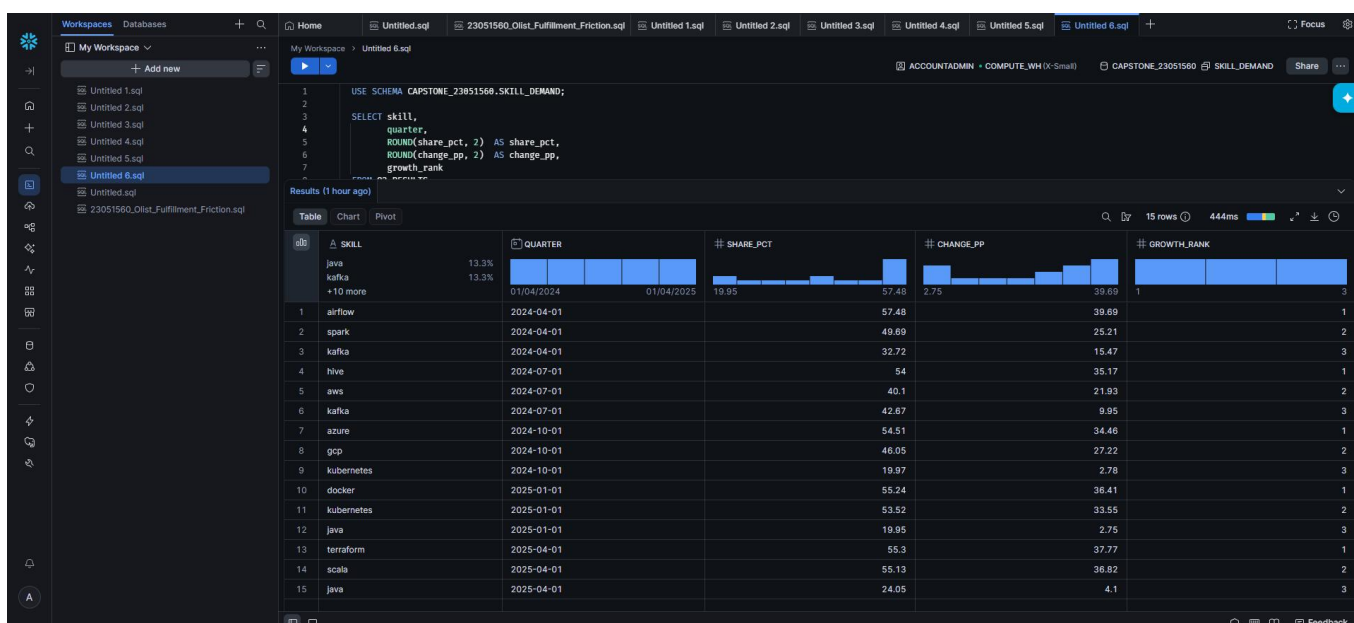


Figure 10.10: Snowflake quarter-on-quarter growth query and verification result.

11. Challenges and Learnings

Challenge	How it was solved
The naive fix looked correct: splitting skills_raw on ',' alone processed all 53,000-odd postings without error.	It silently produced one invented skill for each of the 1,000 comma-joined rows and lost roughly 4,400 real mentions. Fixed by splitting on a comma-or-semicolon character class and re-checking that zero exploded values still contain a comma.
Capitalisation and padding split one skill three ways ('Snowflake', 'SNOWFLAKE', ' Snowflake ') and pushed it below Docker in the ranking.	Normalised each exploded value once (trim, then lowercase) before counting distinct strings, and confirmed 57 raw strings collapse to exactly 20 canonical skills.
Spark 4 rejected the date-offset arithmetic used in the local generator.	Cast the month/day offset explicitly to INT, and kept the original expression alongside the compatible one so the

Challenge	How it was solved
	change is auditable.
Assuming a rerun-safe Snowflake load rather than testing it.	Ran COPY INTO on the same three files a second time and confirmed LOAD_SKIPPED with zero rows added, instead of trusting that COPY INTO is idempotent by default.

Key learning: a pipeline is only as trustworthy as the reconciliation around it. The five-number Silver subtraction, the independent oracle check and the repeat-load test all exist to catch a plausible-looking pipeline that quietly produces the wrong ranking.

12. Future Improvements

- Add controlled snapshot replacement or file-version management for future ingestion cycles, rather than a one-time curated export.
- Add an orchestration layer (e.g. a Snowflake Task or an external scheduler) if unattended Databricks-to-Snowflake integration is ever required.
- Replace the synthetic generator with a real, ethically sourced job-postings dataset with documented permissions if the project needs market-level conclusions.
- Add city, sector and company-size filters to the dashboard so the committee can drill into the same verified Gold tables.

13. Conclusion

Job Postings Skill Demand turns 55,350 messy synthetic job postings into 51,840 clean posting records and 227,624 posting-skill rows. The Gold layer has 360 skill-month rows and 6,660 skill-pair rows, and it answers the three assignment questions: top ten skills per month, fastest quarter-on-quarter growth, and companion skills. The Silver row counts reconcile exactly, and Snowflake passes all 14 full-result checks.

The main lesson is that a pipeline is only as good as the checks around it. A split on a single separator ran without any error but changed the top-ten ranking, and only the reconciliation checks caught it. The Snowflake load is manual and the data is synthetic, so the results describe this dataset and not the live job market.

14. References and Links

Item	Details
Assignment brief	Capstone Project Briefs, Topic 26, pages 104-107 (provided course document)
Dataset	Synthetic job postings generated with seed 26, modelled on the Kaggle LinkedIn Job Postings dataset
GitHub repository	https://github.com/AadiRatn/job-postings-skill-demand
Dashboard	https://aadiratn.github.io/job-postings-skill-demand/
Databricks documentation	input_file_name and file-provenance guidance
Snowflake documentation	COPY INTO <table> and data-load considerations
Project evidence	README, FINAL_VERIFICATION.md and the Snowflake verification SQL included in the submission folder